

Tiny Vision-Language Model that Spells Scenes

Ons Chtioui
RLS Entrance
Challenge
Technical Report

1. Problem

The model must look at a 64x64 image (one or two colored shapes on a plain background) and write, letter by letter, the word describing the scene (size+color+shape, optionally followed by a spatial relation and a second object). The learning goal was to build every component of a minimal Vision-Language Model from scratch, without shortcuts: a visual encoder, hand-written multi-head attention, a causal Transformer decoder, and to honestly measure what the model truly understands about the image rather than chasing a high score alone.

2. Architecture

The pipeline follows exactly the schema required by the brief: image -> CNN encoder -> linear projection (adapter) -> visual tokens -> concatenation with letter tokens -> prefix-masked Transformer decoder -> linear head -> greedy letter-by-letter decoding.

Stage	Tensor shape	Design choice
Image	(B, 3, 64, 64)	-
CNN encoder	(B, 128, 8, 8)	3 Conv2d layers, stride 2, channels 32-64-128, BatchNorm+ReLU
Adapter	(B, 64, 128)	Linear 128 -> d_model, spatial flatten into 64 visual tokens
Full sequence	(B, 64+L, 128)	d_model=128, 4 heads, 2 decoder layers, FFN=256
Logits	(B, L+1, 27)	27 = 26 letters + <eos>

Masking choice (not imposed by the brief): a **prefix mask** - visual tokens attend to each other freely (bidirectional), while letter tokens are causal among themselves but always see the full set of visual tokens. This reflects the idea that the image is a fully known context at the moment each letter is generated.

3. Implementation

Multi-head attention was written entirely by hand (Q/K/V projections, head splitting, scaled dot-product with a $1/\sqrt{d_k}$ factor, additive mask before softmax, head merging, output projection). Its correctness was verified against `F.scaled_dot_product_attention` on the same inputs and mask: maximum observed difference of 6.7e-08 in float32, well below the required 1e-5 threshold.

Loss masking was a subtle point: padding positions are ignored via `ignore_index=-100` in the cross-entropy, using the id as filler on the input side (never meaningfully used since its gradient is cut off on the target side).

Bug found and fixed. An early version had an off-by-one misalignment between logits and targets: the position right after the image predicted the word's second letter instead of its first, and the last letter was never trained to produce . This bug was invisible in the training loss (which stayed low) but made exact-match accuracy zero (0.0%), since the model's output was globally shifted by one position. It was found by manually inspecting a handful of full generated predictions rather than trusting the loss number alone, then fixed by aligning targets to the full letters+eos sequence (length L+1 instead of L).

4. Training Setup

Parameter	Value
Optimizer	AdamW, lr=3e-4
Batch size	32
Epochs (main model)	10

Hardware	Google Colab, NVIDIA Tesla T4 GPU
Seed	fixed seed from generate_data.py (splits unmodified)
Model parameters	~1.2M

5. Experiments

E0 - overfit one batch (bug check). Question: can the pipeline learn at all? Hypothesis: on a single batch, the loss should approach 0 within a few hundred steps. Result: loss went from 3.4512 (step 0) to 0.0088 (step 300, see appendix figure). Interpretation: the full pipeline (data, encoder, attention, decoder, loss) is wired correctly.

E1 - blind baseline. Question: does the model actually use the image, or is it mostly guessing from text statistics? Hypothesis: replacing images with zeros should collapse exact-match if the model truly depended on the image. Same architecture, same training protocol (10 epochs), only the image changes. Result: see table in Section 6. The gap is stark, confirming the hypothesis.

S1 - required throughput benchmark. Question: where does this model's compute cost sit? Images/second was measured at 4 batch sizes (1, 16, 64, 256) on the T4 GPU, after a warm-up phase (5 untimed iterations, since the first iterations pay for cuDNN autotuning) and with torch.cuda.synchronize() called before each timer read (GPU execution is asynchronous). Data loading is excluded from the timing (data generated in memory).

Additional test - 10 vs 15 epochs. Question: does training longer improve the model? Result: at 15 epochs, the final validation loss (0.1968) is markedly worse than at 10 epochs (0.0313), with the true minimum reached earlier, around epoch 10 (val loss 0.0283). Interpretation: the model starts overfitting past 10 epochs; the 10-epoch checkpoint is kept as the main reported result.

6. Results

Experiment	Configuration	Metric	Result
Main model	10 epochs, batch 32	Exact match (test)	68.2%
E1 (blind baseline)	10 epochs, images=0	Exact match (test)	0.0%
E1 (blind baseline)	same	Final val loss	1.3475 (vs 0.0313 normal, ~43x worse)
S1 (throughput)	batch 64, T4 GPU	images/second	5500.6

Per-attribute accuracy (normal model vs. blind baseline)

Attribute	Normal model	Blind baseline
size (object 1)	88.25%	49.75%
color (object 1)	80.55%	30.60%
shape (object 1)	81.40%	24.10%
relation	41.86%	2.55%
size (object 2)	75.78%	4.02%
color (object 2)	59.90%	2.25%
shape (object 2)	61.86%	0.98%

S1 - throughput by batch size (T4 GPU)

Batch size	1	16	64	256
images/second	140.2	2109.0	5500.6	6804.5

Measurements on synthetic in-memory data (excluding disk loading), after 5 warm-up iterations and with explicit CUDA synchronization before timing.

7. Analysis

The gap between the normal model and the E1 blind baseline (0% vs. 68.2% exact-match, validation loss about 43x higher without the image) shows unambiguously that the model genuinely exploits visual content rather than guessing from text-only regularities.

The **relation** attribute (leftof/rightof/above/below) is by far the hardest for the normal model to learn (41.9%, versus 80%+ for color/shape/size of the first object). One hypothesis is that a relation depends on the *relative* position of two objects in the image, a more abstract signal for a simple CNN encoder to extract than the color or shape of a single isolated object; pooling the spatial map down to just 8x8 visual tokens may also dilute the fine-grained positional information needed to tell above/below apart from leftof/rightof.

Attributes of the second object (size2, color2, shape2) are consistently predicted worse than those of the first object, even in the normal model. This is consistent with error accumulation during letter-by-letter decoding: an early mistake in the word can derail generation of the letters that follow, an effect typical of autoregressive generation.

On the blind baseline, the *size* attribute stays at 49.75%, noticeably higher than color (30%) or shape (24%). This has a simple explanation: size has only 2 possible values (small/large), so a model guessing essentially at random from word-length statistics alone already reaches about 50% by chance, whereas color (4 values) and shape (4 values) have a lower chance baseline.

Regarding throughput (S1), moving from batch 64 to batch 256 brings only a modest gain (5500.6 to 6804.5 images/s, i.e. +24% for a 4x larger batch), while moving from 16 to 64 brings a more-than-proportional gain (2109.0 to 5500.6, i.e. x2.6 for a 4x larger batch). This suggests that between batch 64 and 256 the T4 GPU approaches its compute limit (compute-bound) for this model, rather than being limited by kernel-launch overhead, which was more likely the case at smaller batch sizes.

8. Limitations

- Only a single run was performed per configuration (no repetition across seeds), so reported numbers carry no variance estimate, which would be expected at Level 2.
- The S1 benchmark measures the model and optimizer alone, without a real DataLoader or disk loading; actual training cost (including I/O) may differ.
- No GPU memory analysis (peak memory vs. batch size) was performed.
- Evaluation on test-heldout (color-shape combinations never seen during training) was not performed; it would measure the model's compositional generalization, a finer question than exact-match on the standard test split.
- Planned next step: repeat the main training run across 3 seeds to report a mean and standard deviation, and evaluate on test-heldout with a dedicated error analysis.

Reproducibility

Every number in this report can be regenerated with a fresh clone of the repository and the following commands, run in order on a Google Colab T4 GPU runtime:

```
git clone https://github.com/ons-sup/rls-tiny-vlm.git && cd rls-tiny-vlm
pip install Pillow tqdm numpy && python generate_data.py
python -m tests.test_shapes && python -m tests.test_attention
python -m src.train --mode e0
python -m src.train --mode full --epochs 10
python -m src.run_eval --checkpoint checkpoint_epoch9.pt --split test
python -m src.train_blind --epochs 10
python -m src.run_eval --checkpoint checkpoint_blind_epoch9.pt --split test
python -m benchmarks.throughput
```

All random seeds come from the fixed seed baked into generate_data.py (data splits were never modified), library versions are pinned in requirements.txt, and the GPU model (NVIDIA Tesla T4) is recorded for every timing result reported in Section 6, in line with the challenge's reproducibility requirements.

9. References

- Vaswani et al., "Attention Is All You Need", 2017 (multi-head attention mechanism).
- Liu et al., "Visual Instruction Tuning" (LLaVA), 2023 (treating visual tokens as language tokens for a decoder).
- PyTorch documentation: `torch.nn`, `torch.profiler`, `F.scaled_dot_product_attention`.
- RLS Entrance Challenge project brief - Tiny Vision-Language Model.